# MQTT Performance Evaluation: Local and Public Server Behavior

Oscar C. Valderrama-Riveros
*Fac. de Ingeniería*
*Universidad Cooperativa de Colombia*
Ibagué, Colombia
oscar.valderramar@campusucc.edu.co

Fernando G. Tinetti
Fac. de Informática
UNLP, and CIC Prov.Bs. As.
La Plata, Argentina
fernando@info.unlp.edu.ar

***Abstract*—This article presents several important aspects related to the operation of the MQTT (Message Queue Telemetry Transport) protocol. More specifically, it focuses on environment differences in both implementation and performance, particularly in terms of communication latency. A comparative analysis is provided between the use of a local MQTT broker (i.e., within a local network or an environment independent of internet traffic) and a public broker. Although both environments are viable for different applications, differences are observed in MQTT's inherent asynchronous behavior and the communication latency. In particular, it is well known that the use of a public broker necessarily introduces higher latency. This work quantitatively demonstrates the differences in communication times, as well as the variability observed across messages, expressed through statistical measures of latency. This work can be considered a methodological approach for evaluating both implementation and performance when using MQTT in applications deployed either in relatively contained environments—such as a company, home, or building—or in fully geographically distributed scenario involving public brokers and internet traffic.**

***Keywords—Publish/Subscribe Architecture, MQTT Communication Protocol, Communications Performance, Asynchronous Communications.***

## I. INTRODUCTION

MQTT [10] [8] is a well-known protocol widely used in many production systems [2] [13]. It was originally designed to enable communication between processes or computing systems that are geographically distant. Furthermore, MQTT was initially intended to operate over data transmission channels with low quality and reliability [r6]. Today, it is considered one of the main alternatives for M2M (Machine-to-Machine) communications and in IoT (Internet of Things) and IIoT (Industrial Internet of Things) environments, among others [17] [11] [1].

MQTT could be strongly related to a more general and conceptual software architecture definition: publish/subscribe systems (or pub/sub) [6] [15]. The pub/sub systems view is particularly useful in decoupling software and hardware systems, because they focus on events. Events determine the state changes of a system, so the development and conceptual view of the system is based on a) the relevant events, b) processes/hardware in which events are identified/generated, i.e. publishers, and c) process/hardware in which the events are processed, i.e. subscribers. Section IV describes and measures performance in a scenario where the broker and clients include fault tolerance features, some of them defined by the MQTT protocol itself and some other depending on the broker implementation and service configuration.

Under the pub/sub view or (sub)system approach, an event broker (or broker, for short) is responsible for all events communication, i.e. transferring events (data) from publishers to subscribers. Thus, a possibly complex (sub)system is naturally decoupled into processes defined by the events they are related to as publishers, subscribers, or both. Besides, this system architecture approach is particularly useful for distributed systems, because processes and hardware are not required to be in the same physical location, they are said to be decoupled [15]. Simple and clear system definition has led even to implement some parts of non-distributed software systems following the pub/sub approach as long as there are events to be processed.

Pub/Sub architectures impose several requirements to the application programming, being asynchronous processing and uniform and ubiquitous communication system among the two most important ones. More specifically:

- Asynchronous processing: since subscribers are decoupled from publishers, they are not able to know in advance the time at which events will *arrive*. Subscribers necessarily have to be ready to receive and process events at any time.
- Ubiquitous communication system: the broker should be able to communicate with every publisher and subscriber process at any given (and asynchronous) time. Every publisher and subscriber communicate only with the event broker, there is no way of communicating to each other.

As a communication protocol, MQTT not only includes these application requirements but also incorporates a range of additional features required for practical deployment. Figure 1 shows the general diagram of an application that uses MQTT.

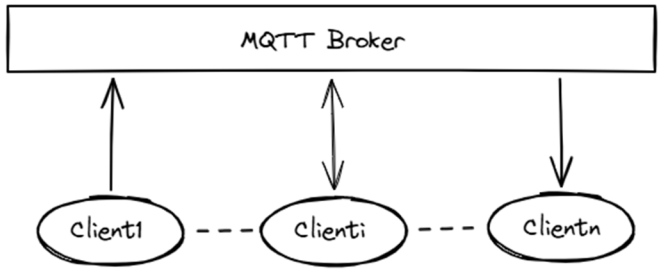



Fig. 1. Schematic MQTT Application.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

Client1 represents a process that publishes MQTT topics, Clientn represents a process that subscribes to MQTT topics, and Clienti represents a process that both publishes and subscribes to MQTT topics.

Figure 2 shows the same general diagram of an application that uses MQTT, highlighting that the information and control flow required for each client to interact with the broker can be bidirectional, regardless of whether the process publishes and/or subscribes to MQTT topics. In particular, when using what MQTT defines as Quality of Service levels 1 or 2 (QoS 1 or QoS 2), the interaction between each client process and the broker becomes relatively complex, as illustrated by the bidirectional dashed arrows in Figure 2. For each topic or data communication (i.e., including *payload*), multiple protocol control messages—defined by the MQTT standard—may be exchanged. The bidirectional dashed arrows represent those control messages.

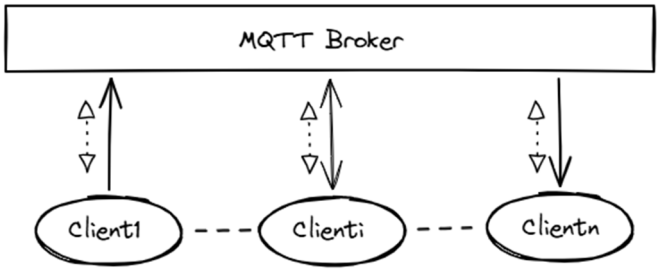



Fig. 2. Schematic MQTT Application: Control Messages.

In particular, MQTT defines three Quality of Service (QoS) levels that specify different guarantees for message delivery (not referred to message performance or latency). QoS 0, also known as *at most once*, delivers messages without requiring any acknowledgment, resulting in minimal overhead but providing no guarantee of delivery. QoS 1, or *at least once*, ensures that messages are delivered by requiring acknowledgments, although duplicates may occur. QoS 2, known as *exactly once*, provides the highest level of reliability by using a multi-step handshake process to ensure that each message is delivered exactly once, at the cost of increased protocol overhead and latency. More specifically, QoS 2 implies a lot of control messages overhead as indicated in Figure 2 with the dashed line bidirectional arrows. The number and type of data and control messages for each QoS level is described in the MQTT Standard and in papers like [3] [12].

It is worth noting that QoS definitions refer to the point-to-point interaction between MQTT clients and MQTT broker. Each MQTT (client) publisher is absolutely independent of every MQTT (client) subscriber. Besides, taking into account QoS and asynchronous communication implies several differences in behavior and latency depending on whether the processes are *nearer* in terms of application distribution.

The next sections include incremental details in MQTT behavior and performance. Section II introduces, with a simple example, important details on MQTT asynchronous behavior that affects the client programming when accessing a broker over the internet. Section III shows the MQTT overhead (rtt: round trip time, measured in a whole virtualized environment) and communication performance (rtt) when accessing a broker over the internet. Section IV describes and measures the overhead when the server and the clients are configured and programmed for handling faults and processes restarts. Section V provides de conclusions of the work introduced and explained in this paper.

## II. Local and Public/Internet Broker

As an initial experiment, a minimal MQTT setup was implemented, consisting of one client publishing a topic and another client subscribing to the same topic. In order to compare both the implementation and the behavior of the clients, one experiment was conducted in a local network environment, while another was performed using the public broker provided by HiveMQ [7].

Figure 3 illustrates the experimental setup, which consists of three machines connected through a local network, all virtualized using Oracle VirtualBox. Although a virtualized environment does not necessarily represent a real-world deployment scenario, it provides a controlled setting to validate both the implementation approach and the behavior of the processes and communications involved in publish/subscribe interactions.

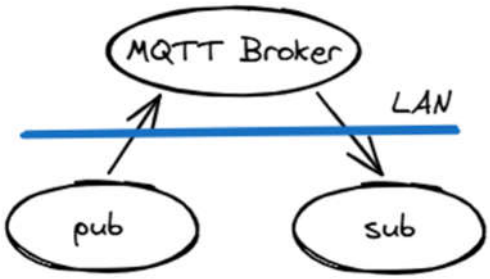



Fig. 3. Simple Virtualized MQTT Environment.

The entire experimental setup was implemented using Eclipse mosquitto, including both the broker and the client library [5]. The first experiment, programmed with the minimum number of functions calls for each client is shown in Figure 4.

```
//Local - Publisher
mosquitto_connect(…);
mosquitto_publish(…);
mosquitto_disconnect(…);

//Local - Subscriber
void on_message(…)
{…}
…
mosquitto_message_callback_set(… on_message);
mosquitto_connect(…);
mosquitto_subscribe(…);
mosquitto_loop_forever(…);
```

Fig. 4. Mosquitto MQTT Publisher and Subscriber, Local Experiments.

In Figure 4, the publisher follows a simple sequence, as it could be expected:

(a) Establishing a connection with the broker,

(b) Publishing a topic, and

(c) Terminating the connection.

The subscriber, on the other hand, explicitly has to "register" the function to be used when the topic is received and enters a "forever loop", in which the library checks whether a topic is received and, in that case, the "registered" function is called/used.

The entire virtualized environment deployed was run in a desktop computer and QoS 0 was used in both clients. Every topic published was received by the subscriber. Once again, the environment is completely contained in the host computer, so there is no more limitation than the host computer resources. The experiment shows that every process is running successfully.

Given that a fully operational MQTT client setup has been established, the next step is to evaluate the same clients using a public broker, which involves communication over the Internet. For this purpose, the publicly available HiveMQ MQTT broker is used. Figure 5 shows the experimental setup with the HiveMQ broker, enabling comparison of both client implementation and execution behavior. While no differences are expected from a conceptual or functional perspective, implementation and performance differences are anticipated, as shown in the following sections.

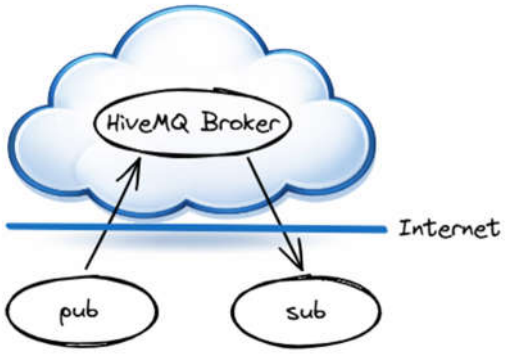



Fig. 5. HiveMQTT Broker in the MQTT Environment.

Figure 6 shows how the clients were implemented to operate with the public HiveMQ broker. As expected, there are some differences with respect to the implementation shown in Figure 4, while other implementation details are less straightforward to anticipate. The process with the minimum—and, in fact, expected—changes is the subscriber.

When comparing the subscriber in Figure 6 with that in Figure 4, three additional functions can be identified, labeled as (1), (2), and (3) in Figure 6. Functions (1) and (2) are associated with the provision of authentication and transport security parameters. Function (3) is analogous to that used in the local environment, but configured with the HiveMQ broker URL.

Actually, functions (1) to (3) have to be used, by every HiveMQTT client for security and privacy issues, given that the HiveMQ environment provides services on the cloud/internet. Thus, these functions are used in the publisher as well as in the subscriber. However, the publisher in Figure 6 also has functions identified with (4), (5), and (6), which were not used in the publisher in Figure 4. Furthermore, no topic arrives to the subscriber unless functions (4) to (6) are used. Whenever a published topic does not arrive to a subscriber, a programmer first attempt would be to use QoS 1 and 2, but it does not solve the problem either.

```
// Public/HiveMQ Subscriber
void on_message(…)
{…}
…
mosquitto_message_callback_set(…);
mosquitto_username_pw_set(…);      // (1)
mosquitto_tls_set(…);              // (2)
mosquitto_connect(…);              // (3)
mosquitto_subscribe(…);
mosquitto_loop_forever(…);

// Public/HiveMQ Publisher
mosquitto_username_pw_set(…);      // (1)
mosquitto_tls_set(…);              // (2)
mosquitto_connect(…);              // (3)
mosquitto_publish(…);
mosquitto_loop_start(mosq);        // (4)
sleep(1);                          // (5)
mosquitto_loop_stop(…);            // (6)
mosquitto_disconnect(…);
```

Fig. 6. Mosquitto MQTT Publisher and Subscriber, Local Experiments.

The underlying problem by which the publisher of Figure 6 has to use something like functions (4) to (6) in Figure 6 is that the library is as asynchronous as the entire MQTT protocol. If the process publishes a topic and immediately closes the connection, the topic does not arrive to the broker. The library is not able to immediately send the topic to the broker given that the broker is "far away", so the topic publication is retained in local queues to be sent to the broker as soon as possible. Looking in detail, functions (4) to (6) require to start a runtime thread for the library to check control and data messages exchanged with the MQTT server, plus a delay of 1s before that runtime thread is terminated. It is not the most elegant solution, but it works: every published topic is received by the subscriber, even using QoS 0. This simple experiment shows that even a single QoS 0 publication, with a little payload (less than 10 bytes) requires the exchange of data and control messages mentioned when explaining Figure 2 above.

## III. Performance Evaluation I

Eclipse mosquitto provides both MQTT client tools and the client library/API used in this work (as shown in Figures 4 and 6). The mosquitto clients are fully functional MQTT client implementations, enabling the development of complete MQTT-based applications. In publish/subscribe systems, as in many application domains, performance must be considered in addition to functionality. Thus, specific experiments are conducted to analyze and compare performance.

Well-known ping-pong experiments [3] are implemented with both, MQTT clients provided by Eclipse mosquitto and MQTT clients implemented in C language, calling mosquitto functions library. These experiments allow to compare performance as well as other timing parameters such as

minimum and maximum communication times. Ping-pong experiment results are expected to reflect the behavior and performance in a local and *controlled* environment as well as in a real geographically- distributed system. Then, *partial* measurements or focus on specific MQTT event timings as in [16] are not taken into account in this paper. Other approaches, such as comparing with other protocols, and/or adding middleware [14] are discarded, this paper is focused on the MQTT protocol itself.

Figure 7 schematically shows the ping-pong experiments, with two processes, named ping and pong respectively. In particular, the ping process starts the cycle by sending of a "ping" topic and, after receiving the "pong" topic is able to calculate the rtt (round-trip time).

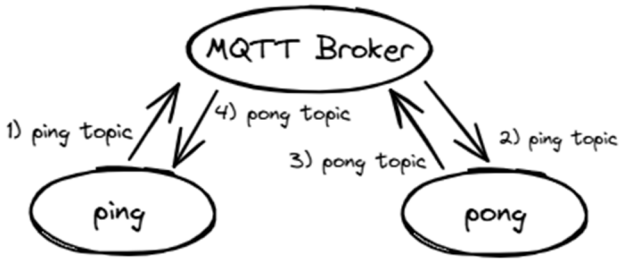



Fig. 7. Ping-Pong Experiment.

The sequence of messages between the broker clients and the broker server for a ping-pong experiment is shown in Figure 7. Given the nature of the distributed computing environment as well as the asynchronous MQTT behavior, the timestamps for computing the communication times have to be taken in a single computer unless there is clock synchronization, which is usually not available in systems where MQTT is used.

The time required for a whole round-trip time (rtt) of the ping-pong experiment accounts for the sequence:

| Ping Process | Pong Process | Pong Process | Ping Process |
|---|---|---|---|
| 1)publication$_1$ | 2)subscription$_1$ | 3)publication$_2$ | 4)subscription$_2$ |

A timestamp $t_1$ is taken before the "ping topic" publication, identified as "1)…" in Figure 7, and another timestamp, $t_2$, is taken after receiving the "pong topic" identified as "4)…" in Figure 7. The rtt is then calculated as rtt = $t_2$-$t_1$.

Just as an example of Eclipse mosquitto client's usage simplicity, Figure 8 shows the "pong" process (as shown in Figure 7 above) of the ping-pong experiments.

```
mosquitto_sub -h $BROKER -t ping -q 2 | while read MSG
do
   # The same payload is sent back
   mosquitto_pub -h $BROKER -t pong -m "$MSG" -q 2
done
```

Fig. 8. Ping-Pong Experiment: Pong Script Using Eclipse mosquitto Clients.

More specifically, the script in Figure 8 uses the Mosquitto client `mosquitto_sub` for receiving "ping" topics. For each "ping" topic the script publishes a "pong" topic using the other MQTT client: `mosquitto_pub`. In the script, $BROKER is the MQTT broker URL, the received payload is sent back via the "pong" topic, and QoS 2 is used for both topic subscription and publication. The script of Figure 7 could be identified as the "pong" process.

Table I shows the ping-pong timings in the simple virtualized environment described in Figure 3. It is worth noting that there is almost no communication overhead or resource contention in the virtualized environment. Clearly, the C client provides much better performance than the clients provided by Eclipse mosquitto. Results in Table I provide motivation to proceed with the subsequent experiments implemented in C. An intermediate alternative would be to implement the clients in Python; however, since Python is an interpreted language, C is expected to offer better performance for MQTT client implementations.

TABLE I. RTT: C VS MOSQUITTO CLIENT, VIRTUAL ENVIRONMENT

| | ***C Client*** | ***Mosquitto Client*** |
|---|---|---|
| Avg rtt (ms) | 5.6 | 35.3 |
| Min rtt (ms) | 3.0 | 12.0 |
| Max rtt (ms) | 19.5 | 68.0 |

## IV. PERFORMANCE EVALUATION II

In production systems such as home automation or industrial process control, the server is typically located within the local network. In geographically distributed applications, or in systems requiring external data logging and analysis, MQTT topics must be managed by external brokers. Accordingly, the same ping-pong experiment was conducted using the broker provided by HiveMQ. Table II presents the results of the ping-pong experiments using the same clients but with the broker provided by HiveMQ. The additional time introduced by message exchange with the broker is significantly higher than in the fully virtualized case, as expected. Considering the results in Table I (corresponding to the clients implemented in C) and those shown in Table II, the following observations can be made:

- The overhead inherent to MQTT is relatively low (on the order of a few milliseconds), even when including broker-side processing.
- Communication times in an environment involving access to an Internet-based service, specifically HiveMQ, are on the order of several hundred milliseconds. It is worth noting that, in this case, no additional processing is performed at the client side beyond that required for handling MQTT topics.

TABLE II. RTT: HIVEMQ BROKER

| | |
|---|---|
| Avg rtt (ms) | 760 |
| Min rtt (ms) | 750 |
| Max rtt (ms) | 870 |

Table I and Table II show that for clients programmed in C, the minimum and average rtt are very similar. The small difference implies that most of the rtt values are near the

minimum one. It is worth noting that *a priori* there is no relationship among MQTT rtt and hard real-time systems. In particular, MQTT QoS is exclusively related to communication reliability, not about communication time. Actually, for hard real-time systems it is expected to use QoS 2 but that QoS level is the worst one in terms of time, processing, and MQTT control messages overhead. Moreover, the use of Internet-based communication, subject to resource contention and dynamic routing conditions, makes the system unsuitable for hard real-time applications [9] [4].

## V. Performance Evaluation III

The MQTT standard includes several features that can be leveraged to implement fault tolerance. In particular, the mechanisms for establishing connections or sessions with the server, as well as the option of maintaining persistent sessions, are well suited to handling client failures. Furthermore, brokers may implement session persistence using durable storage, thereby enabling the broker itself to tolerate failures such as computer restarts.

MQTT brokers cannot be tolerant to all possible failures, as faults are inherently unpredictable. In particular, the broker provided by Eclipse mosquitto supports recovery after restarts and can resume operation, provided that the underlying file system—and, specifically, the storage used for session persistence—remains uncorrupted following the failure. To take full advantage of the fault tolerance capabilities of MQTT using Eclipse mosquitto, the following modifications were introduced to both the broker configuration and the ping-pong client implementations:

- On the broker side/configuration, persistence was enabled, ensuring that session state is stored in the filesystem. Filesystem usage for persistence is an Eclipse mosquitto broker feature, not required by the MQTT standard.
- Persistent sessions were configured at the client level, allowing clients to resume subscriptions and message exchange after reconnection. In particular, MQTT v5 is requested and session expiry interval is set.
- Automatic client reconnection mechanisms were implemented, enabling recovery from transient failures without manual intervention.
- Clean session (clean start) parameters were adjusted, preventing the loss of session state upon disconnection. Furthermore, QoS 2 was selected, providing message delivery guarantees under failure conditions.
- Error handling and reconnection logic were incorporated into the client code, improving robustness against communication interruptions.

All of those changes in server configuration and ping-pong processes programming involve overheads in computing as well as storage operations. Ping-pong experiments were carried out in order to measure the overhead time implied by those changes for fault handling and tolerance.

Table III shows the ping-pong rrt in the whole virtualized environment configured and programmed for fault tolerance. It is worth noting that the experiments were conducted without the injection of faults; the objective was to evaluate the overhead introduced by the (server) configuration and (pub/sub clients) programming required for handling faults/restarts.

Results in Table III seems to be better than the *comparable* ones in Table I (C Client column), but differences are minimal. In particular, the Max rtt (ms) could have a relatively large difference (19.5 vs 8.9 in this case) but it is likely due to the non-real-time underlying environment and ms (small timing) measurements.

TABLE III. RTT: MOSQUITTO BROKER, TOLERANCE FAILURE OVERHEAD

| | |
|---|---|
| Avg rtt (ms) | 5.4 |
| Min rtt (ms) | 2.0 |
| Max rtt (ms) | 8.9 |

## VI. Conclusions

This paper has shown how the asynchronous nature of the MQTT standard, together with the implementation provided by Eclipse mosquitto, can introduce communication issues in topic delivery that are not immediately apparent during application development. In particular, as discussed in Section II, it is necessary to account for this asynchronous behavior during implementation; otherwise, messages may fail to reach their destination, even when QoS 2 is used.

Section III introduced what can be considered the overhead associated with the transfer of information using MQTT topics. By maintaining the entire experimental setup in a virtualized environment, topics handling (payload and control data) does not involve resource contention on communication channels, nor do they rely on physical network hardware. Additionally, Section III highlights both the simplicity provided by the Eclipse mosquitto clients and the overhead associated with their use.

Section IV presented a simple approach to evaluating the cost of accessing an MQTT server or broker that provides service over an Internet connection. Furthermore, it shows that, in relative terms, the use of both MQTT standard features and broker-side implementations for fault tolerance does not necessarily introduce significant overhead.

From an application programming perspective, it may be beneficial to design synchronous interaction mechanisms with brokers, or at least interfaces that do not hide important operational details, as highlighted in this work. Additionally, the development of simplified mechanisms for fault tolerance would be of interest, both for establishing connections with brokers and for handling failures during runtime.

## References


[1] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A survey on enabling technologies, protocols, and applications," IEEE Communications Surveys & Tutorials, vol. 17, no. 4, pp. 2347–2376, 2015, doi: 10.1109/COMST.2015. 2444095.

[2] M. Bandyopadhyay and A. Bhattacharyya, "Lightweight internet protocols for web enablement of sensors using constrained gateway devices," Computer Communications, vol. 91–92, pp. 86–95, 2016.

[3] D. Borsatti, W. Cerroni, F. Tonini, C. Raffaelli, "From IoT to Cloud: Applications and Performance of the MQTT Protocol", International Conference on Transparent Optical Networks, July 2020, http://dx.doi.org/10.1109/ICTON51198.2020.9203167

[4] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. Springer, 2011.

[5] Eclipse Foundation, Eclipse mosquitto, 2024. [Online]. Available: https://mosquitto.org/

[6] P. T. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, "The many faces of publish/subscribe," ACM Computing Surveys, vol. 35, no. 2, pp. 114–131, 2003.

[7] HiveMQ Inc, "HiveMQ – Stream Data. Build Intelligence. Activate AI", https://www.hivemq.com/agent/

[8] ISO/IEC, Information technology—Message Queuing Telemetry Transport (MQTT) v3.1.1, ISO/IEC 20922:2016, 2016.

[9] H. Kopetz and W. Steiner, *Real-Time Systems: Design Principles for Distributed Embedded Applications*, 3rd ed. Cham, Switzerland: Springer, 2022.

[10] OASIS, MQTT Version 5.0, A. Banks, E. Briggs, K. Borgendale, and R. Gupta, Eds., Mar. 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html

[11] G. Patti, L. Leonardi, G. Testa, L. Lo Bello, PrioMQTT: A prioritized version of the MQTT protocol, Computer Communications, Vol. 220, 2024, pp. 43-51, ISSN 0140-3664, https://doi.org/10.1016/j.comcom.2024.03.018.

[12] N. Saha, P. Paul, K. Ji, R. Harik, "Performance evaluation framework of MQTT client libraries for IoT applications in manufacturing", Manufacturing Letters, Volume 41, Supplement, 2024, ISSN 2213-8463, https://doi.org/10.1016/j.mfglet.2024.09.150.

[13] A. Stanford-Clark and H. L. Truong, "MQTT For Sensor Networks (MQTT-S)," IBM DeveloperWorks, 2013.

[14] D. Thangavel, X. Ma, A. Valera, H.-X. Tan, and C. K.-Y. Tan, "Performance evaluation of MQTT and CoAP via a common middleware," IEEE Ninth International Conference on Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP), 2014.

[15] M. van Steen and A.S. Tanenbaum, Distributed Systems, 4th ed., distributed-systems.net, 2023.

[16] G. Wang, "The Study of IoT MQTT Publish Latency," 2021 IEEE International Conference on Electro Information Technology (EIT), Mt. Pleasant, MI, USA, 2021, pp. 027-031, doi: 10.1109/EIT51626.2021.9491838.

[17] K. Zachariadis, F. Psomakelis, and K. Kolomvatsos, "A survey on IoT protocols: MQTT, CoAP, and REST," Future Internet, vol. 12, no. 6, 2020.